

Similitud Semantica vs. Coincidencia de Palabras Clave

Evaluacion de un modelo de embeddings para discernir
relevancia semantica en el concepto de
"Despliegue de microservicios"

AI Engineering Course - Coderhouse

Ejercicio de la unidad (no es el entregable final)

1. Objetivo

Evaluar la capacidad de un modelo de embeddings para diferenciar entre:

- Relevancia semantica real: oraciones que hablan del mismo concepto tecnico con vocabulario distinto (deberian tener alta similitud coseno).
- Coincidencia superficial de palabras clave: oraciones que comparten terminos lexicos pero no el significado (deberian tener baja similitud coseno pese a compartir palabras).

Concepto tecnico elegido: Despliegue de microservicios.

2. Conjunto de oraciones

2.1 Oraciones semanticamente relevantes (mismo concepto, vocabulario distinto)

#	Oracion	Vocabulario clave
1	Publicamos cada componente de la aplicacion de forma independiente en contenedores dentro del cluster de Kubernetes.	contenedores, cluster, Kubernetes
2	El equipo automatizo la entrega continua de los servicios distribuidos mediante pipelines de CI/CD hacia el entorno productivo.	CI/CD, pipelines, productivo
3	Cada modulo del sistema se empaqueta como una imagen Docker y se libera de manera autonoma sin afectar a los demas.	Docker, imagen, autonoma
4	La orquestacion de los servicios se realiza escalando replicas dinamicamente segun la demanda en la nube.	orquestacion, replicas, escalado
5	Lanzamos versiones nuevas de cada microservicio usando estrategias de rollout progresivo (canary) sin downtime.	rollout, canary, downtime

2.2 Oraciones trampa (comparten palabras clave, significado opuesto o irrelevante)

#	Oracion	Motivo de la trampa
T1	El servicio de micro-limpieza es excelente y llega puntual todos los martes a la oficina.	Comparte el prefijo "micro-" y la palabra "servicio", pero no tiene relacion con software.
T2	El despliegue de las tropas se realizo en la frontera tras la orden del comando militar.	Comparte la palabra "despliegue", pero el contexto es militar, no tecnico.

2.3 Resultado esperado

Un buen modelo de embeddings deberia producir:

- Similitud coseno ALTA (aprox. 0.6 - 0.9) entre las oraciones 1-5 entre si.
- Similitud coseno BAJA (aprox. 0.0 - 0.3) entre las oraciones 1-5 y las oraciones trampa T1/T2, a pesar de la coincidencia lexica parcial ("micro-", "despliegue").

Esto demuestra que el modelo captura el significado contextual y no solo la superposicion de tokens, a diferencia de un enfoque lexico (ej. TF-IDF o coincidencia exacta de palabras).

3. Calculo de Similitud Coseno con scikit-learn

scikit-learn provee la funcion `cosine_similarity` dentro del modulo `sklearn.metrics.pairwise`, que recibe matrices de vectores (embeddings) y devuelve la matriz de similitudes por pares.

Formula: $\text{cos_sim}(A, B) = (A \cdot B) / (\|A\| * \|B\|)$

El resultado va de -1 (opuestos) a 1 (identicos en direccion), siendo 0 la ortogonalidad (sin relacion).

3.1 Ejemplo en Python

```
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

# 1. Embeddings ya generados por un modelo (ej. text-embedding-3-small)
embedding_oracion_1 = np.array([[0.12, 0.08, -0.03]]) # vector oracion 1
embedding_oracion_2 = np.array([[0.11, 0.07, -0.02]]) # vector oracion 2

# 2. Similitud coseno entre ambos vectores
similitud = cosine_similarity(embedding_oracion_1, embedding_oracion_2)
print(f"Similitud coseno: {similitud[0][0]:.4f}")

# 3. Comparar una oracion base contra un conjunto de candidatas
def obtener_embedding(texto: str) -> list[float]:
    # Placeholder: aqui se llamaria al proveedor real
    # (ej. client.embeddings.create(model="text-embedding-3-small", input=texto))
    ...

oracion_base = "Publicamos cada componente en contenedores dentro de Kubernetes."
candidatas = [
    "El equipo automatizo la entrega continua mediante CI/CD.",
    "El servicio de micro-limpieza es excelente y llega puntual.",
]

vector_base = np.array([obtener_embedding(oracion_base)])
vectores_candidatas = np.array([obtener_embedding(c) for c in candidatas])
similitudes = cosine_similarity(vector_base, vectores_candidatas)

for oracion, score in zip(candidatas, similitudes[0]):
    print(f"[{score:.4f}] {oracion}")
```

3.2 Notas de implementacion

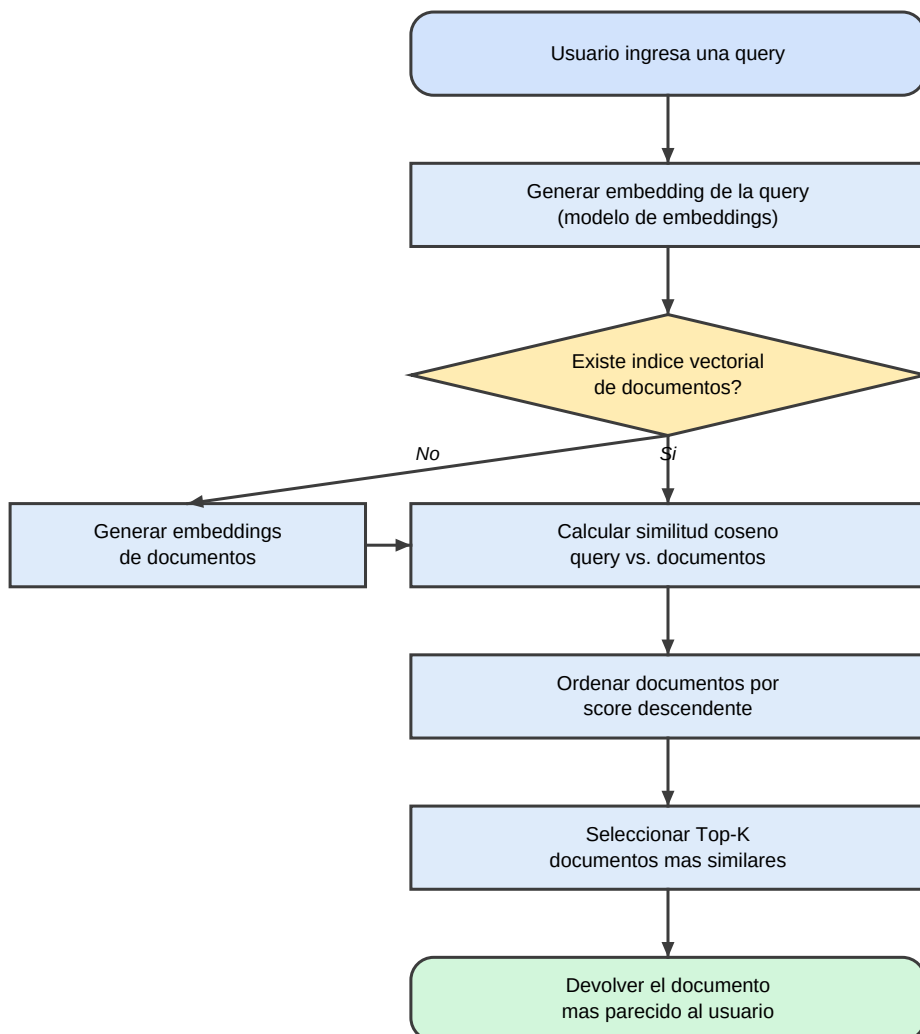
- `cosine_similarity` acepta matrices de forma $(n_muestras, n_dimensiones)$; un solo vector debe pasarse como `[...]` o usando `.reshape(1, -1)`.
- Para busqueda semantica conviene calcular los embeddings de los documentos una sola vez y

compararlos contra la query en una sola llamada vectorizada.

- Si los embeddings estan normalizados (norma $L2 = 1$), la similitud coseno equivale al producto punto simple, mas rapido de calcular.

4. Diagrama de flujo: proceso de busqueda semantica

Equivalente visual del diagrama Mermaid definido en el archivo .md del ejercicio:



5. Conclusion

El ejercicio evidencia que la similitud coseno sobre embeddings permite capturar relevancia semantica real: las oraciones 1-5, pese a no compartir vocabulario, deberian agruparse por significado, mientras que las oraciones trampa T1/T2, pese a compartir palabras, deberian quedar claramente separadas por un score bajo. Esta es la base conceptual de los sistemas de busqueda semantica (RAG, recuperacion de documentos, etc.), que superan las limitaciones de la busqueda lexica tradicional (keyword matching).